

Unicode와 UTF-8 간단히 이해하기

Down Jeong

Follow

4 min read · Jan 1, 2019

139

2

13

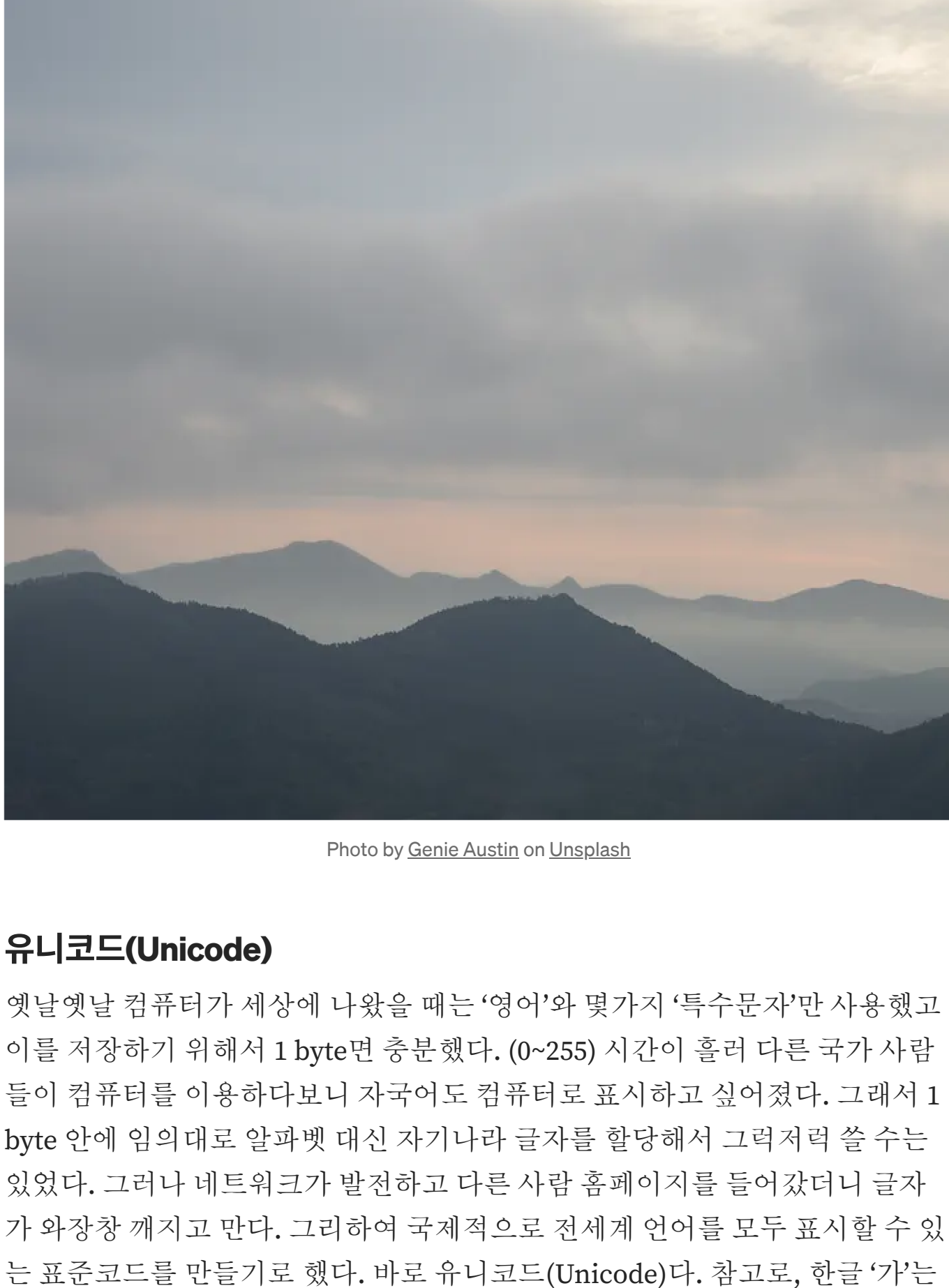


Photo by Scott Austin on Unsplash

유니코드(Unicode)

옛날 옛날 컴퓨터가 세상에 나왔을 때는 ‘영어’와 몇 가지 ‘특수문자’만 사용했고 이를 저장하기 위해서 1 byte만 충분했다. (0-255) 시간이 흘러 다른 국가 사람들이 컴퓨터를 이용하다보니 자국어도 컴퓨터로 표시하고 싶어졌다. 그래서 1 byte 안에 입체로 완제품 대신 자키나와 글자를 할당해서 그려쳐다 볼 수는 있었다. 그러나 네트워크가 발전하고 다른 사람 홈페이지를 들어갔더니 글자가 이상할 게 되고 만다. 그리하여 국제적으로 일체제 연어를 모두 표시할 수 있는 표준코드를 만들기로 했다. 바로 유니코드(Unicode)다. 한글로, 한글 ‘가’는 유니코드로 ‘U+AC00’이다. 왜 그리나고 이유는 문자마라, 약속이다.

유니코드는 글자와 코드가 1:1 대응되어 있는 ‘코드포인’이다. (코드포인)

UTF-8

유니코드를 통해 코드표가 정의되었다. 남은 것은 그 ‘코드’가 컴퓨터에 어떻게 저장되어야 하는 것이다. 다른 말로 인코딩(encoding)이라고 하는데, 컴퓨터가 이해할 수 있는 형태로 바꿔주는 것이다. 예를 들어, 길동이가 자기만의 암호제계를 만들었는데 ‘가’는 0001(+1)로, ‘나’는 0010(+2)이라고 하자. 길동이 ‘가’나 ‘나’가라고 말하면 컴퓨터에는 0001 0010 0010 0001로 저장한다. 글자당 1 byte로 할당해 4 byte로 저장할 수 있고 00010010 00100001처럼 2byte로 저장할 수도 있다. 이런 원칙으로 길동이와 컴퓨터간의 약속이다. 길동이만 쓸 때는 문제가 없는데 전세계 사람들이 사용하려면 유 더 잘 만들어야 할 것이다.

UTF-8은 유니코드를 인코딩(encoding)하는 방식이다. 현재 제에서 사용하는 약 4이다.

UTF-8은 가변 인코딩방식이다. 쉬운 말로 하면 글자마다 byte 길이가 다르다는 것이다. ‘a’는 1 byte이고 ‘가’는 3 byte이다. 가변을 구분하기 위해 첫 바이트에 표식을 넣었는데 2 byte는 110으로 시작하고 3바이트는 1110으로 시작한다. 나머지 바이트는 10으로 시작한다. 왜 그리나고 통는다면, 그냥 ‘아축이더’라고 말하고 싶다.

코드포인 범위	범위	첫 바이트	둘째 바이트	셋째 바이트	넷째 바이트	다섯째 바이트	여섯째 바이트
7비트	0-127	0xxxxxxx					
11비트	128-2,047	11xxxxxx	10xxxxxx				
16비트	2,048-65,535	111xxxxx	10xxxxxx	10xxxxxx			
21비트	65,536-2,097,151	1111xxxx	10xxxxxx	10xxxxxx	10xxxxxx		
26비트		11111xxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31비트		111111xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

출처: 위키미디어

코딩

Go를 이용하면 다른 프로그래밍 언어보다 좀 더 직관적으로 확인할 수 있다. Go는 에디터에서 작성한 모든 문자열을 UTF-8로 인코딩하는데 이 문자열을 받 복문으로 처리할 때 첫 번째 인자로 글자 시작 byte의 index를 가져온다. 타언어들의 경우, 문자열에 대한 글자의 index를 가져오는 데 Go는 이 부분을 보다 명확하게 한 것이다. 두 번째 인자는 int32 (혹은 rune이라 코딩도 한다) 타입의 실제 인코딩된 데이터이다.

```
package main
import "fmt"
func main() {
    word := "abc12345"
    for i, r := range word {
        fmt.Printf("%d %d %d %c\n", i, r, r, r)
    }
    fmt.Println()
    for i := 0; i < len(word); i++ {
        fmt.Printf("%d(%c) ", word[i], word[i])
    }
}
```

```
0 97 8481 a
1 98 8482 b
2 99 8483 c
3 48482 84848 1
4 48288 848488 1
5 48288 848484 1
97(a) 98(b) 99(c) 234(4) 176(1) 128(c) 235(4) 138(1) 152( ) 235(4) 138( ) 164(=)
```

상당 코라

알파벳의 경우, UTF-8 인코딩 시 1 byte를 차지하므로 1씩 증가한다. 하지만 한글의 경우, 3 byte를 차지하므로 3씩 증가한다. int32는 총 4 byte로 유니코드의 모든 문자를 표현한다. 재미있는 건, word[i]로 접근하게 되면 uint8 타입의 데이터가 나오는데 바로 byte이다. 그래서 한글의 경우, 1 byte씩 읽게 되면 ASCII 코드를 읽게 된다.

Get Down Jeong's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

유니코드는 국제표준 문자표이고 UTF-8은 인코딩 방식이다. 구글링 하다면 이에 대한 굉장히 많은 자료가 있지만 오히려 그 방대한 내용때문에 기본적인 개념이 헷갈리기도 한다. 헷갈린 사람에게 조금이나마 도움이 되었으면 좋겠다.

Unicode

UTF-8

GoLang

139

2

13

Written by Down Jeong

41 followers · 9 following

A Gopher living in the Java world. Working at Toss Securities. Previously: Coupang, Hyperconnect, SK Telecom, LG Electronics, Pantech

Follow

Responses (2)

Write a response

What are your thoughts?

Felix You

Nov 4, 2020

...

좋은 글 감사합니다. 그런데 헷갈리는 게 있어서 질문 드립니다.
'유니코드는 가변 인코딩방식이다.' 라고 유니코드가 아니라 UTF-8에 아닐까요?

5

Back

David Jung

Oct 14, 2020

...

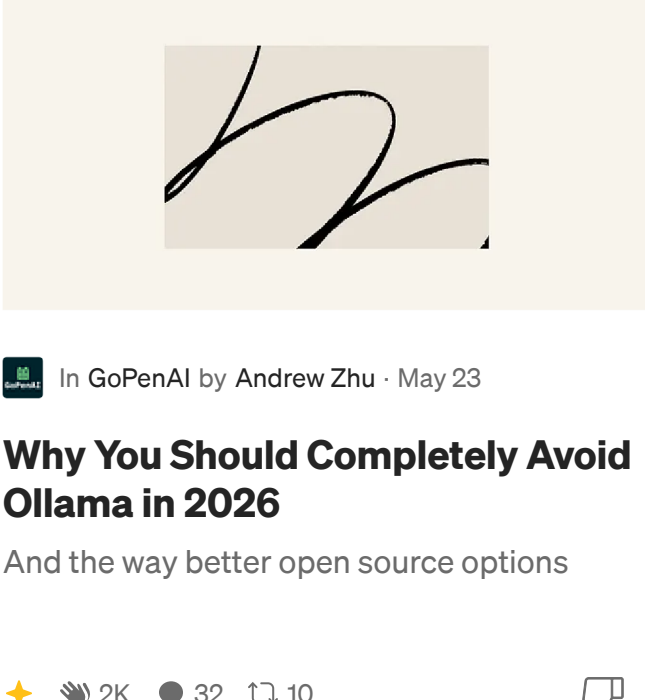
잘 보고 갑니다.

Back

More from Down Jeong



Readability
코드를 잘 한다는 것은 무엇일까? 나는 가독성이 좋은 코드를 만드는 것이라 생각한다. 그럼 가독...



Simplicity
나는 단순한 것을 좋아한다.



프로그래밍 언어를 공부하면 무엇이 좋은가?
프로그래밍 언어는 그다지 중요하지 않다는 말에 반론 동의하고 반론 동의할 수 없다. 진짜는 소스코드...



다수 원소(Majority Element) 찾기
어떤 배열에 절반수 이상 존재하는 원소가 있다면 그 원소를 찾고 싶다. 어떻게 찾을 수 있을까? 예...

See all from Down Jeong

Recommended from Medium



As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...
Most people do #1 within 10 minutes of waking (and it sabotages your entire day)



7 Coding Patterns I Stole From Senior Engineers
Most developers do not become better because they learn more syntax. They...



How Did Islam Spread So Fast?
In the 600s AD, Mecca was a village of a few hundred people, off the beaten path, a...



Stop Memorizing Design Patterns: Use This Decision Tree Instead
Choose design patterns based on pain points: apply the right pattern with minimal over...



Kubernetes is Officially Doomed (And Linux Torvalds Warned Us)
Why tech giants are quietly abandoning the orchestration king, and the \$10 million...



Why You Should Completely Avoid Oilman in 2026
And the way better open source options

See more recommendations